# Bereitstellen der Anwendung

NetApp Solutions

NetApp
January 10, 2023

This PDF was generated from https://docs.netapp.com/de-de/netapp-solutions/ai/mlrun_get_code_from_github.html on January 10, 2023. Always check docs.netapp.com for the latest.

# Inhaltsverzeichnis

Bereitstellen der Anwendung . . . . . 1
Code von GitHub . . . . . 1
Konfigurieren Sie Die Arbeitsumgebung . . . . . 2
Implementierung Von Grafana Dashboard . . . . . 13

# Bereitstellen der Anwendung

In den folgenden Abschnitten wird die Installation und Bereitstellung der Anwendung beschrieben.

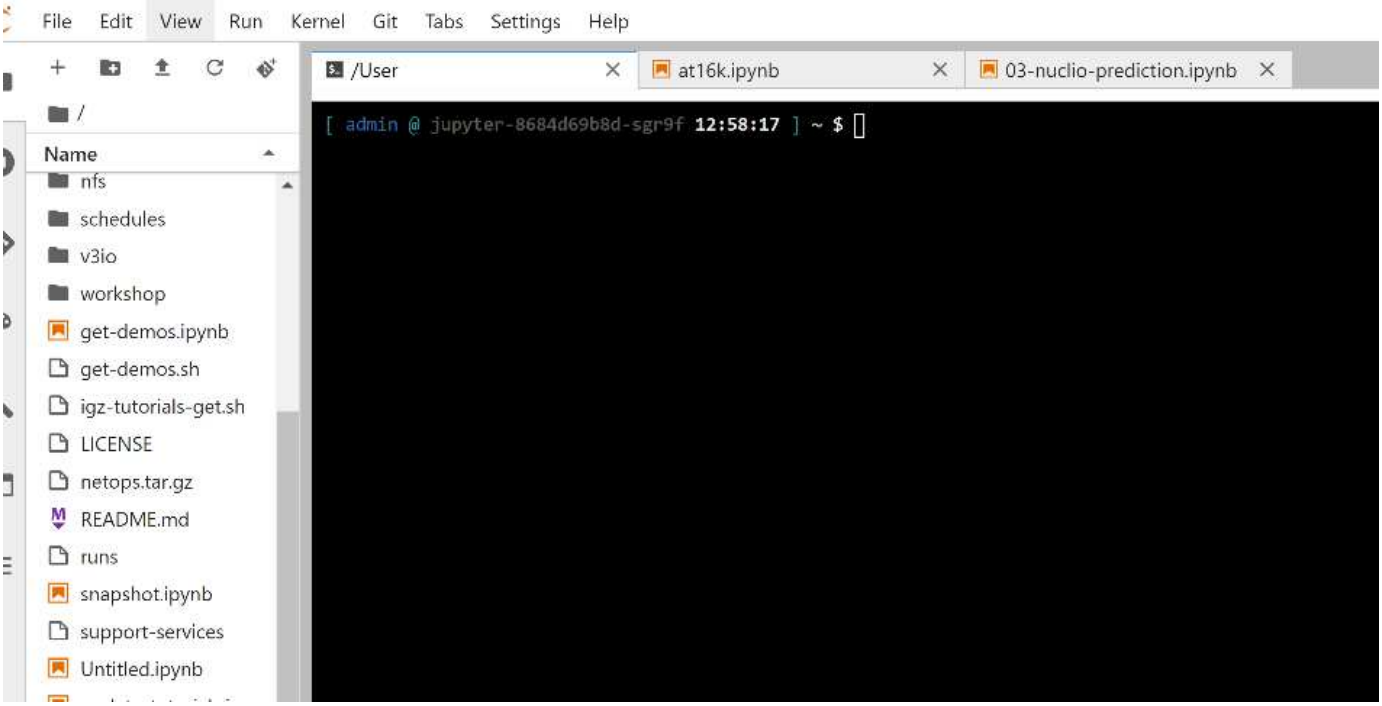
"Als Nächstes erhalten Sie Code von GitHub".

## Code von GitHub

Nachdem das NetApp Cloud Volume oder NetApp Trident Volume nun für den Iguazio Cluster und die Entwicklungsumgebung zur Verfügung steht, können Sie nun mit der Überprüfung der Applikation beginnen.

Benutzer haben ihren eigenen Arbeitsbereich (Verzeichnis). Auf jedem Notebook befindet sich der Pfad zum Benutzerverzeichnis `/User`. Die Iguazio-Plattform verwaltet das Verzeichnis. Wenn Sie die obigen Anweisungen befolgen, steht das NetApp Cloud Volume im zur Verfügung `/netapp` Verzeichnis.

Holen Sie sich den Code von GitHub mit einem Jupyter Terminal.




Klont das Projekt in der Jupyter-Terminalaufforderung.

```
cd /User
git clone .
```

Sie sollten jetzt sehen, die `netops- netapp` Ordner in der Dateistruktur im Jupyter-Arbeitsbereich.

"Als Nächstes: Arbeitsumgebung Konfigurieren"

# Konfigurieren Sie Die Arbeitsumgebung

Kopieren Sie die `Notebook set_env-Example.ipynb` Als `set_env.ipynb`. Öffnen und bearbeiten `set_env.ipynb`. Dieses Notizbuch legt Variablen für Anmeldeinformationen, Dateipositionen und Ausführungstreiber fest.

Wenn Sie die obigen Anweisungen befolgen, sind die folgenden Schritte die einzigen Änderungen, die vorgenommen werden müssen:

1. Erhalten Sie diesen Wert über das Iguazio Services Dashboard: `docker_registry`

   Beispiel: `docker-registry.default-tenant.app.clusterq.iguaziodev.com:80`

2. Ändern `admin` An Ihren Iguazio-Benutzernamen:

   ```
   IGZ_CONTAINER_PATH = '/users/admin'
   ```

   Nachstehend finden Sie Einzelheiten zur Systemverbindung mit ONTAP. Geben Sie den Volume-Namen an, der bei der Installation von Trident generiert wurde. Die folgende Einstellung gilt für ein ONTAP Cluster vor Ort:

   ```
   ontapClusterMgmtHostname = '0.0.0.0'
   ontapClusterAdminUsername = 'USER'
   ontapClusterAdminPassword = 'PASSWORD'
   sourceVolumeName = 'SOURCE VOLUME'
   ```

   Die folgende Einstellung gilt für Cloud Volumes ONTAP:

```
MANAGER=ontapClusterMgmtHostname
svm='svm'
email='email'
password=ontapClusterAdminPassword
weid="weid"
volume=sourceVolumeName
```

## Basis-Docker-Images Erstellen

Alles, was Sie zum Aufbau einer ML-Pipeline benötigen, ist in der Iguazio-Plattform enthalten. Der Entwickler kann die Spezifikationen der Docker-Images festlegen, die für die Ausführung der Pipeline erforderlich sind, und die Image-Erstellung aus Jupyter Notebook ausführen. Öffnen Sie das Notebook `create-images.ipynb` Und alle Zellen ausführen.

Dieses Notizbuch erstellt zwei Bilder, die wir in der Pipeline verwenden.

- `iguazio/netapp.` Zum behandeln VON ML-Aufgaben.

### Create image for training pipeline

```
[4]: fn.build_config(image=docker_registry+'/iguazio/netapp', commands=['pip install \
     v3io_frames fsspec>=0.3.3 PyYAML==5.1.2 pyarrow==0.15.1 pandas==0.25.3 matplotlib seaborn yellowb
     fn.deploy()
```

- `netapp/pipeline`. Enthält Dienstprogramme zur Bearbeitung von NetApp Snapshot Kopien.

### Create image for Ontap utilitites

```
[9]: fn.build_config(image=docker_registry + '/netapp/pipeline:latest',commands=['apt -y update','pip install v3io_frames netapp_ontap'
     fn.deploy()
```

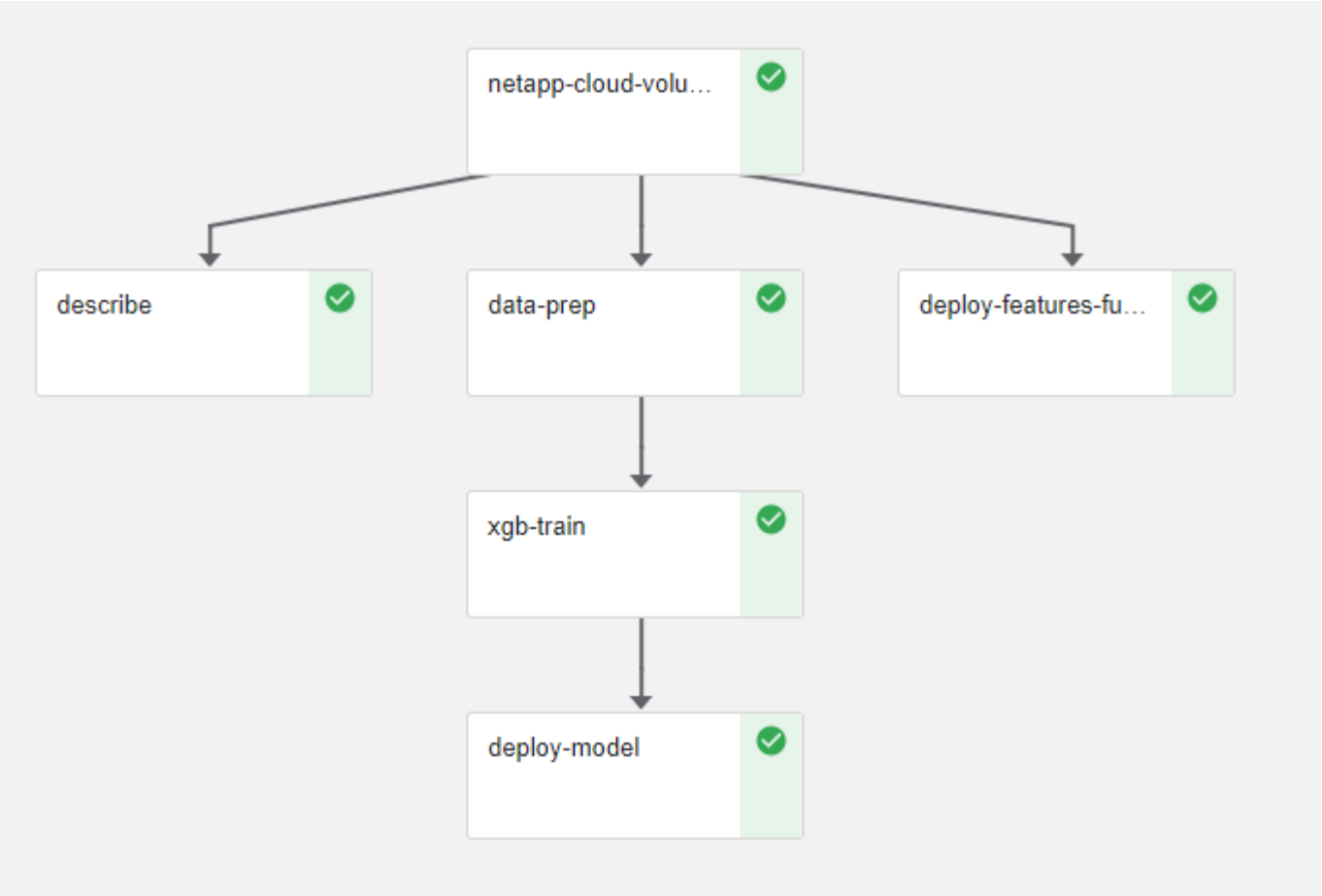
## Überprüfen Sie Einzelne Jupyter Notebooks

In der folgenden Tabelle sind die Bibliotheken und Frameworks aufgeführt, mit denen wir diese Aufgabe erstellt haben. All diese Komponenten wurden vollständig in die rollenbasierten Zugriffs- und Sicherheitskontrollen von Iguazio integriert.

| Bibliotheken/Framework | Beschreibung |
|---|---|
| MLRun | Ein von Iguazio verwaltet, um die Montage, Ausführung und Überwachung einer ML/AI-Pipeline zu ermöglichen. |
| Nuclio | Ein serverloses Funktionsgerüst, das mit Iguazio integriert ist. Auch als Open-Source-Projekt von Iguazio verfügbar. |
| Kubeflow | Kubernetes-basiertes Framework für die Implementierung der Pipeline Dies ist auch ein Open-Source-Projekt, zu dem Iguazio beiträgt. Es ist in Iguazio integriert für zusätzliche Sicherheit und Integration in den Rest der Infrastruktur. |
| Docker | Eine Docker-Registry wird als Service in der Iguazio-Plattform ausgeführt. Sie können dies auch ändern, um eine Verbindung zu Ihrer Registrierung herzustellen. |
| NetApp Cloud Volumes | Cloud Volumes auf AWS ermöglichen uns Zugriff auf große Datenmengen und die Möglichkeit, Snapshot Kopien mit den für das Training verwendeten Datensätzen zu erstellen. |
| Trident | Trident ist ein Open-Source-Projekt, das von NetApp gemanagt wird. Sie vereinfacht die Integration in Storage- und Computing-Ressourcen in Kubernetes. |

Für die Erstellung der ML-Pipeline haben wir mehrere Notebooks verwendet. Jedes Notebook kann einzeln getestet werden, bevor es in die Pipeline gebracht wird. Wir decken jedes Notebook individuell nach dem Bereitstellungsablauf dieser Demonstrationsanwendung ab.

Das gewünschte Ergebnis ist eine Pipeline, mit der ein Modell auf Basis einer Snapshot-Kopie der Daten trainiert und das Modell zur Inferenz implementiert wird. Im folgenden Bild ist ein Blockdiagramm einer abgeschlossenen MLRun-Pipeline dargestellt.




## Implementierung Der Funktion Zur Datengenerierung

In diesem Abschnitt wird beschrieben, wie wir Nuclio serverlose Funktionen zur Generierung von Netzwerkgerätedaten verwendet haben. Der Anwendungsfall wird von einem Iguazio-Client aus angepasst, der die Pipeline implementierte und Iguazio-Services zur Überwachung und Vorhersage von Fehlern bei Netzwerkgeräten verwendete.

Wir haben Daten von Netzwerkgeräten simuliert. Ausführen des Jupyter-Notebooks `data-generator.ipynb` Erstellt eine serverlose Funktion, die alle 10 Minuten läuft und eine Parkett-Datei mit neuen Daten erzeugt. Um die Funktion bereitzustellen, führen Sie alle Zellen in diesem Notebook aus. Siehe "Nuclio Website" Um unbekannte Komponenten in diesem Notebook zu überprüfen.

Beim Generieren der Funktion wird eine Zelle mit dem folgenden Kommentar ignoriert. Jede Zelle im Notebook wird als Teil der Funktion angenommen. Importieren Sie das Nuclio-Modul, um es zu aktivieren `%nuclio magic`.

```
# nuclio: ignore
import nuclio
```

In der Spezifikation für die Funktion definieren wir die Umgebung, in der die Funktion ausgeführt wird, wie sie ausgelöst wird und welche Ressourcen sie verbraucht.

```
spec = nuclio.ConfigSpec(config={"spec.triggers.inference.kind":"cron",

"spec.triggers.inference.attributes.interval" :"10m",
                                "spec.readinessTimeoutSeconds" : 60,
                                "spec.minReplicas" : 1},……
```

Der `init_context` Die Funktion wird beim Initialisieren der Funktion vom Nuclio Framework aufgerufen.

```
def init_context(context):
    ….
```

Jeder Code, der nicht in einer Funktion enthalten ist, wird aufgerufen, wenn die Funktion initialisiert wird. Wenn Sie ihn aufrufen, wird eine Handler-Funktion ausgeführt. Sie können den Namen des Handlers ändern und in der Funktionungsspezifikation angeben.

```
def handler(context, event):
            …
```

Sie können die Funktion vor der Bereitstellung vom Notebook aus testen.

```
%%time
# nuclio: ignore
init_context(context)
event = nuclio.Event(body='')
output = handler(context, event)
output
```

Die Funktion kann über das Notebook bereitgestellt oder über eine CI/CD-Pipeline (Anpassung dieses Codes) bereitgestellt werden.

```
addr = nuclio.deploy_file(name='generator',project='netops',spec=spec,
tag='v1.1')
```

#### Pipeline-Notebooks

Diese Notizbücher sollen für dieses Setup nicht einzeln ausgeführt werden. Dies ist nur ein Bericht über jedes Notizbuch. Wir haben sie als Teil der Pipeline aufgerufen. Um sie einzeln auszuführen, lesen Sie die MLRun Dokumentation durch, um sie als Kubernetes Jobs auszuführen.

#### Snap_cv.ipynb

Dieses Notebook wickelt die Snapshot-Kopien des Cloud Volumes am Anfang der Pipeline ab. Er übergibt den Namen des Volumes an den Pipeline-Kontext. Dieses Notebook stößt ein Shell-Skript an, um die Snapshot

Kopie zu bearbeiten. Während der Ausführung in der Pipeline enthält der Ausführungskontext Variablen, um alle Dateien zu finden, die für die Ausführung benötigt werden. Beim Schreiben dieses Codes muss sich der Entwickler nicht um den Speicherort der Datei im Container kümmern, der ihn ausführt. Wie später beschrieben, wird diese Applikation mit allen Abhängigkeiten implementiert und ist die Definition der Pipeline-Parameter, die den Kontext der Ausführung liefern.

```
command = os.path.join(context.get_param('APP_DIR'),"snap_cv.sh")
```

Der Speicherort der erstellten Snapshot-Kopie wird im MLRun-Kontext platziert, der von Schritten in der Pipeline verwendet werden soll.

```
context.log_result('snapVolumeDetails',snap_path)
```

Die nächsten drei Notebooks werden parallel ausgeführt.

#### Data-prep.ipynb

RAW-Metriken müssen in Funktionen für das Modelltraining umgewandelt werden. Dieses Notebook liest die RAW-Kennzahlen aus dem Snapshot Verzeichnis und schreibt die Funktionen für das Modelltraining auf das NetApp Volume.

Bei Ausführung im Kontext der Pipeline ist der Input `DATA_DIR` Enthält den Speicherort der Snapshot Kopie.

```
metrics_table = os.path.join(str(mlruncontext.get_input('DATA_DIR',
os.getenv('DATA_DIR','/netpp'))),
                            mlruncontext.get_param('metrics_table',
os.getenv('metrics_table','netops_metrics_parquet')))
```

#### Beschreiben.ipynb

Um die eingehenden Kennzahlen zu visualisieren, stellen wir einen Pipeline-Schritt bereit, der über die Kubeflow und MLRun UIs verfügbare Plots und Diagramme bereitstellt. Jede Ausführung hat eine eigene Version dieses Visualisierungstools.

```
ax.set_title("features correlation")
plt.savefig(os.path.join(base_path, "plots/corr.png"))
context.log_artifact(PlotArtifact("correlation",  body=plt.gcf()),
local_path="plots/corr.html")
```

#### Deploy-Feature-function.ipynb

Wir überwachen kontinuierlich die Metriken, die nach Anomalien suchen. Dieses Notizbuch erstellt eine serverlose Funktion, die die Funktionen generiert, die für die Vorhersage von eingehenden Metriken erforderlich sind. Dieses Notizbuch ruft die Erstellung der Funktion auf. Der Funktionscode befindet sich im Notebook `data- prep.ipynb`. Beachten Sie, dass wir für diesen Zweck dasselbe Notebook wie einen Schritt in der Pipeline verwenden.

#### Training.ipynb

Nachdem wir die Funktionen erstellt haben, lösen wir das Modelltraining aus. Das Ergebnis dieses Schritts ist das Modell, das zur Inferenz verwendet wird. Wir sammeln auch Statistiken, um jede Ausführung (Experiment) im Auge zu behalten.

Beispielsweise gibt der folgende Befehl die Genauigkeitbewertung in den Kontext des entsprechenden Experiments ein. Dieser Wert ist in Kubeflow und MLRun sichtbar.

```
context.log_result('accuracy',score)
```

#### Bereitstellen-Inferenz-Funktion.ipynb

Der letzte Schritt in der Pipeline ist die Implementierung des Modells als serverlose Funktion für kontinuierliche Inferenz. Dieses Notebook ruft die Erstellung der serverlosen Funktion auf, die in definiert ist `nuclio-inference- function.ipynb`.

## Pipeline prüfen und aufbauen

Die Kombination der Ausführung aller Notebooks in einer Pipeline ermöglicht es, kontinuierlich Experimente durchzuführen, um die Genauigkeit des Modells anhand neuer Metriken neu zu bewerten. Öffnen Sie zuerst das `pipeline.ipynb` Notebook. Wir erläutern Ihnen die Details, die zeigen, wie NetApp und Iguazio die Implementierung dieser ML-Pipeline vereinfachen.

Wir verwenden MLRun, um jedem Schritt der Pipeline Kontext bereitzustellen und die Ressourcenzuordnung zu bearbeiten. Der MLRun API-Service wird auf der Iguazio-Plattform ausgeführt und dient als Interaktionsstelle mit Kubernetes-Ressourcen. Jeder Entwickler kann keine Ressourcen direkt anfordern. Die API verarbeitet die Anforderungen und ermöglicht Zugriffskontrollen.

```
# MLRun API connection definition
mlconf.dbpath = 'http://mlrun-api:8080'
```

Diese Pipeline kann mit NetApp Cloud Volumes und On-Premises-Volumes eingesetzt werden. Wir haben diese Demo für die Verwendung von Cloud Volumes erstellt. Im Code sehen Sie jedoch die Option zur Ausführung vor Ort.

```
# Initialize the NetApp snap fucntion once for all functions in a notebook
if [ NETAPP_CLOUD_VOLUME ]:
    snapfn =
code_to_function('snap',project='NetApp',kind='job',filename="snap_cv.ipyn
b").apply(mount_v3io())
    snap_params = {
    "metrics_table" : metrics_table,
    "NETAPP_MOUNT_PATH" : NETAPP_MOUNT_PATH,
    'MANAGER' : MANAGER,
    'svm' : svm,
    'email': email,
    'password': password ,
    'weid': weid,
    'volume': volume,
    "APP_DIR" : APP_DIR
       }
else:
    snapfn =
code_to_function('snap',project='NetApp',kind='job',filename="snapshot.ipy
nb").apply(mount_v3io())
….
snapfn.spec.image = docker_registry + '/netapp/pipeline:latest'
snapfn.spec.volume_mounts =
[snapfn.spec.volume_mounts[0],netapp_volume_mounts]
      snapfn.spec.volumes = [ snapfn.spec.volumes[0],netapp_volumes]
```

Die erste Aktion, die erforderlich ist, um ein Jupyter-Notebook in einen Kubeflow-Schritt zu verwandeln, ist, den Code in eine Funktion zu verwandeln. Eine Funktion verfügt über alle Spezifikationen, die zum Ausführen dieses Notebooks erforderlich sind. Wenn Sie das Notebook nach unten scrollen, sehen Sie, dass wir für jeden Schritt in der Pipeline eine Funktion definieren.

| Teil des Notebooks | Beschreibung |
|---|---|
| <Code_to_Function> (Teil des MLRun-Moduls) | Name der Funktion: Projektname. Dient zur Organisation aller Projektartefakte. Dies ist in der MLRun UI sichtbar. Freundlich. In diesem Fall ist Kubernetes-Job. Dies könnte DASK, mpi, funkk8s und mehr sein. Weitere Informationen finden Sie in der MLRun-Dokumentation. Datei: Der Name des Notebooks. Dies kann auch ein Speicherort in Git (HTTP) sein. |
| Bild | Der Name des Docker Images, das wir für diesen Schritt verwenden. Das haben wir früher mit dem Notebook create-image.ipynb erstellt. |
| Volume_Mounts und Volumes | Einzelheiten zum Mounten des NetApp Cloud Volume zur Laufzeit. |

Außerdem definieren wir Parameter für die Schritte.

```
params={   "FEATURES_TABLE":FEATURES_TABLE,
           "SAVE_TO" : SAVE_TO,
           "metrics_table" : metrics_table,
           'FROM_TSDB': 0,
           'PREDICTIONS_TABLE': PREDICTIONS_TABLE,
           'TRAIN_ON_LAST': '1d',
           'TRAIN_SIZE':0.7,
           'NUMBER_OF_SHARDS' : 4,
           'MODEL_FILENAME' : 'netops.v3.model.pickle',
           'APP_DIR' : APP_DIR,
           'FUNCTION_NAME' : 'netops-inference',
           'PROJECT_NAME' : 'netops',
           'NETAPP_SIM' : NETAPP_SIM,
           'NETAPP_MOUNT_PATH': NETAPP_MOUNT_PATH,
           'NETAPP_PVC_CLAIM' : NETAPP_PVC_CLAIM,
           'IGZ_CONTAINER_PATH' : IGZ_CONTAINER_PATH,
           'IGZ_MOUNT_PATH' : IGZ_MOUNT_PATH
            }
```

Nachdem Sie die Funktionsdefinition für alle Schritte erstellt haben, können Sie die Pipeline erstellen. Wir verwenden den `kfp` Modul, um diese Definition zu erstellen. Der Unterschied zwischen der Verwendung von MLRun und dem Selbstaufbau besteht in der Vereinfachung und Verkürzung der Codierung.

Die von uns definierten Funktionen werden mit dem in Schrittkomponenten umgewandelt `as_step` Funktion von MLRun.

#### Definition Von Snapshot-Schritten

Initiieren einer Snapshot-Funktion, -Ausgabe und Mounten von v3io als Quelle:

```
snap = snapfn.as_step(NewTask(handler='handler',params=snap_params),
name='NetApp_Cloud_Volume_Snapshot',outputs=['snapVolumeDetails','training
_parquet_file']).apply(mount_v3io())
```

| Parameter | Details |
|---|---|
| NewTask | NewTask ist die Definition der Funktion Run. |
| (MLRun-Modul) | Handler. Name der zu aufrufe Python-Funktion. Wir haben den Namen Handler im Notebook verwendet, aber er ist nicht erforderlich. Param. Die Parameter, die wir an die Ausführung übergeben haben. Innerhalb unseres Codes verwenden wir context.get_param (‘PARAMETER’), um die Werte zu erhalten. |

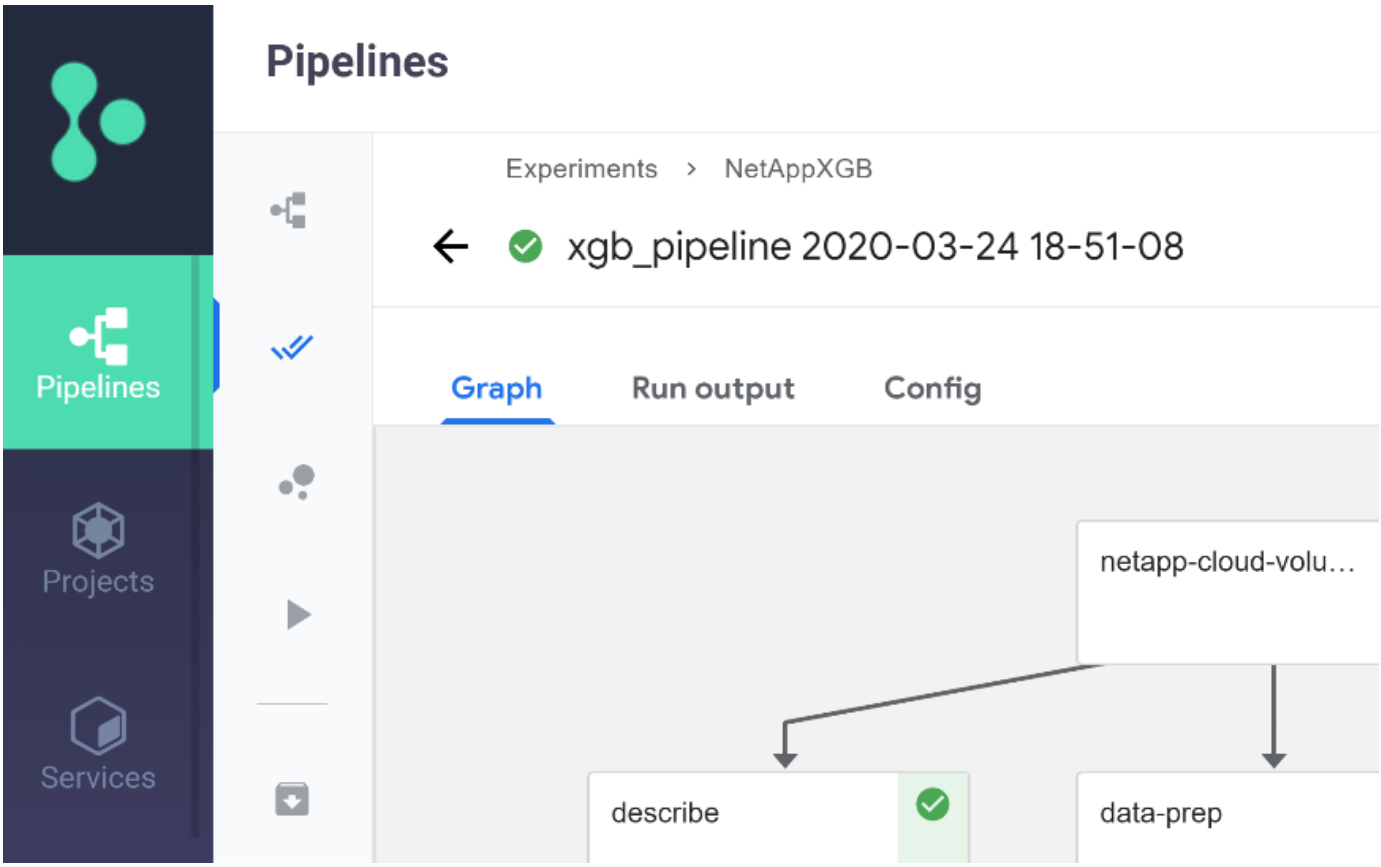
| Parameter | Details |
| --- | --- |
| As_Step | Name: Name des Kubeflow-Pipeline-Schritts. Ausgänge. Dies sind die Werte, die der Schritt dem Wörterbuch nach Abschluss hinzugefügt hat. Werfen Sie einen Blick auf das Snap_cv.ipynb Notebook. Mount_v3io(). Hiermit wird der Schritt zum Mounten von /Benutzer für den Benutzer konfiguriert, der die Pipeline ausführt. |

```
prep = data_prep.as_step(name='data-prep',
handler='handler',params=params,
                          inputs = {'DATA_DIR':
snap.outputs['snapVolumeDetails']} ,

out_path=artifacts_path).apply(mount_v3io()).after(snap)
```

| Parameter | Details |
| --- | --- |
| Eingänge | Sie können die Ausgänge eines vorherigen Schritts an einen Schritt übergeben. In diesem Fall ist Snap.Outputs ['snapVolumeDetails'] der Name der Snapshot Kopie, die wir im Snap Schritt erstellt haben. |
| Out_PATH | Ein Speicherort für Artefakte, die mithilfe des MLRun-Moduls log_Artefakte erzeugt werden. |

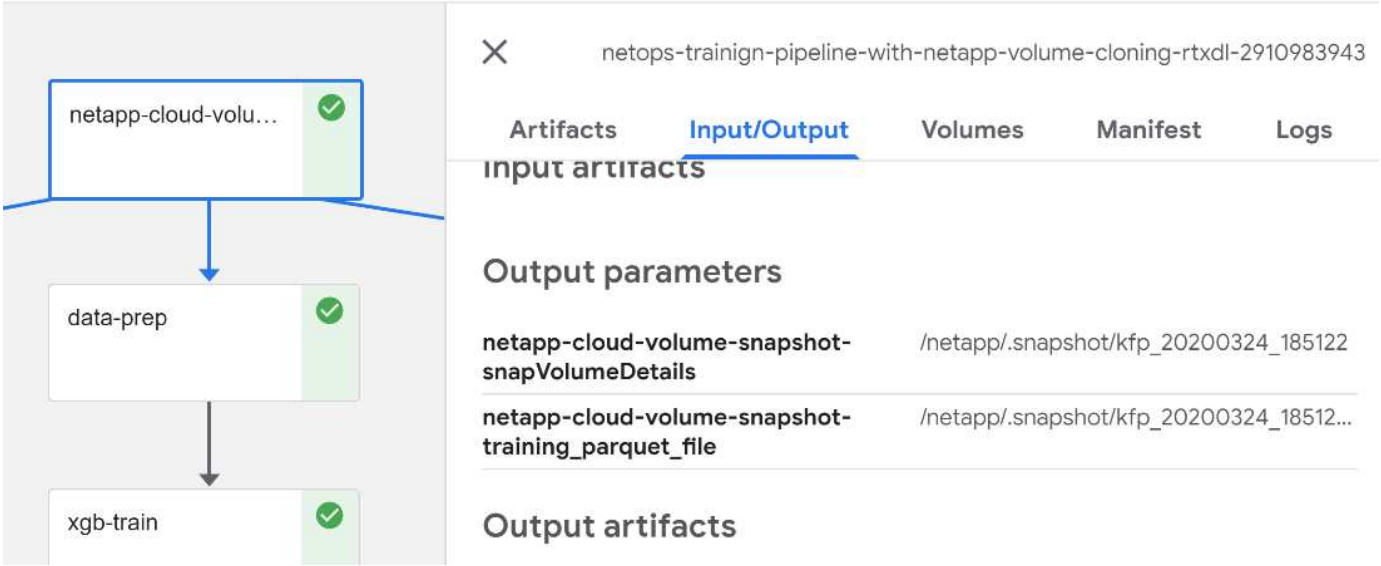
Sie können laufen `pipeline.ipynb` Von oben nach unten. Anschließend können Sie im Iguazio-Dashboard zur Registerkarte Pipelines wechseln, um den Fortschritt zu überwachen, wie auf der Registerkarte Iguazio-Dashboard-Pipelines zu sehen ist.



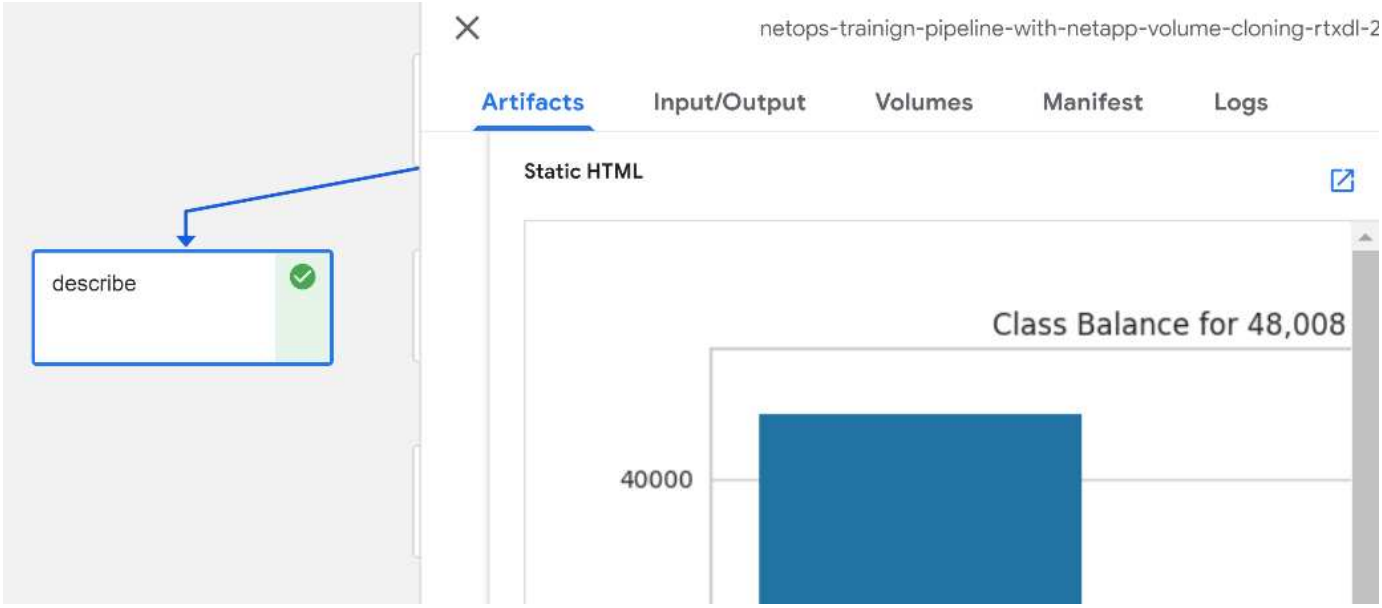

Da wir die Genauigkeit des Trainingsschritts in jedem Lauf protokolliert haben, haben wir für jedes Experiment eine Aufzeichnung der Genauigkeit, wie in der Aufzeichnung der Trainingsgenauigkeit zu sehen ist.

| Run name | Status | Duration | Pipeline Version | Recurring ... | Start time | accuracy |
|---|---|---|---|---|---|---|
| xgb_pipeline 2020-03-24 18-51-... | ✓ | 0:08:43 | [View pipeline] | - | 3/24/2020, 2:51:09 PM | 0.985 |
| xgb_pipeline 2020-03-19 13-31-... | ✓ | 0:08:14 | [View pipeline] | - | 3/19/2020, 9:31:19 AM | 0.980 |
| xgb_pipeline 2020-03-18 12-56-... | ✓ | 0:08:11 | [View pipeline] | - | 3/18/2020, 8:56:08 AM | 0.990 |
| xgb_pipeline 2020-03-17 19-49-... | ✓ | 0:08:03 | [View pipeline] | - | 3/17/2020, 3:49:31 PM | 0.985 |
| xgb_pipeline 2020-03-17 18-34-... | ✓ | 0:05:54 | [View pipeline] | - | 3/17/2020, 2:34:56 PM | 0.980 |
| xgb_pipeline 2020-03-17 17-34-... | ✓ | 0:04:48 | [View pipeline] | - | 3/17/2020, 1:34:16 PM | 0.982 |
| xgb_pipeline 2020-03-17 17-01-... | ✓ | 0:05:25 | [View pipeline] | - | 3/17/2020, 1:01:58 PM | 0.987 |
| xgb_pipeline 2020-03-16 16-47-... | ✓ | 0:06:08 | [View pipeline] | - | 3/16/2020, 12:47:19 ... | 0.983 |
| xgb_pipeline 2020-03-16 13-57-... | ✓ | 0:05:18 | [View pipeline] | - | 3/16/2020, 9:57:03 AM | 0.980 |

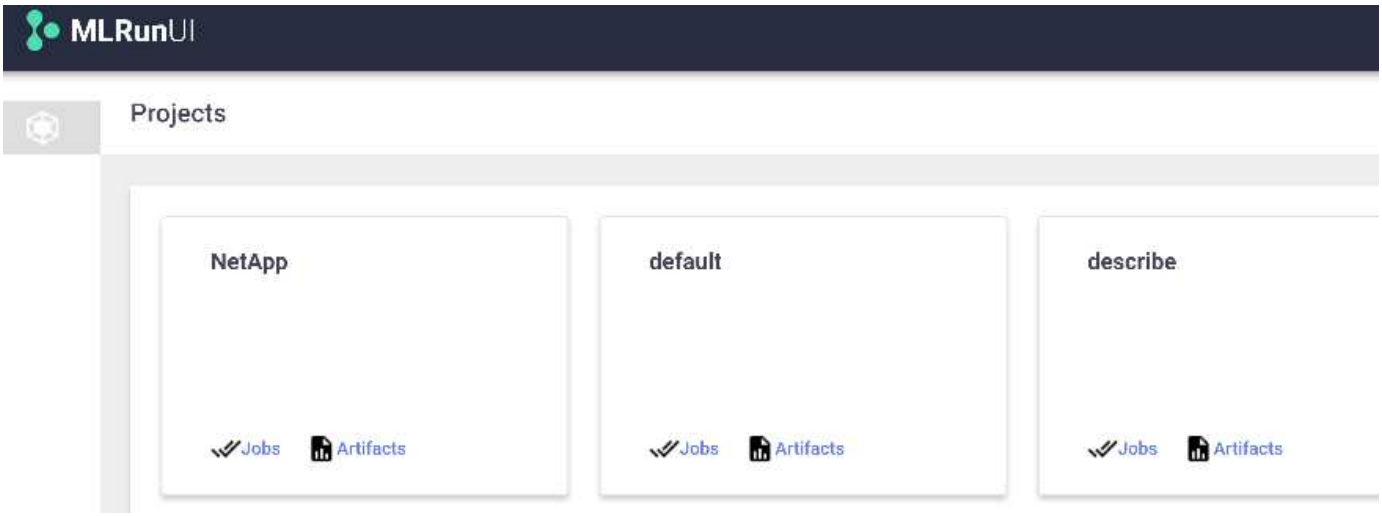
Wenn Sie den Schritt Snapshot auswählen, wird der Name der Snapshot Kopie angezeigt, die zur Ausführung dieses Experiments verwendet wurde.



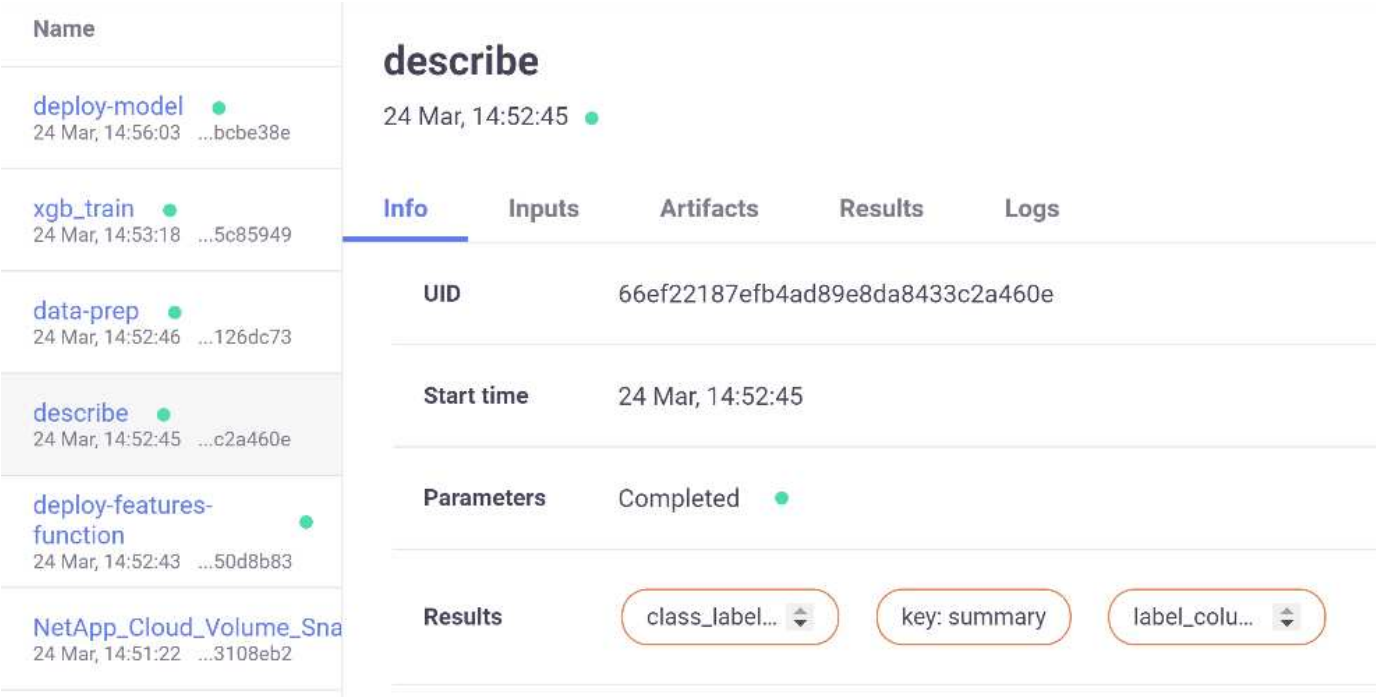

Der beschriebene Schritt hat visuelle Artefakte, um die von uns verwendeten Metriken zu untersuchen. Sie können erweitern, um die vollständige Darstellung wie im folgenden Bild zu sehen.




Die MLRun API-Datenbank verfolgt auch Eingänge, Ausgänge und Artefakte für jeden nach Projekt organisierten Durchlauf. Ein Beispiel für Eingänge, Ausgänge und Artefakte für jeden Durchlauf ist im folgenden Bild zu sehen.




Für jede Aufgabe werden zusätzliche Details gespeichert.




Es gibt mehr Informationen über MLRun, als wir in diesem Dokument abdecken können. AI Artefakte, einschließlich der Definition der Schritte und Funktionen, können in der API-Datenbank gespeichert, versioniert und einzeln oder als volles Projekt aufgerufen werden. Projekte können auch gespeichert und zur späteren Verwendung an Git geschoben werden. Wir empfehlen Ihnen, weitere Informationen im zu erhalten "MLRun GitHub-Website".

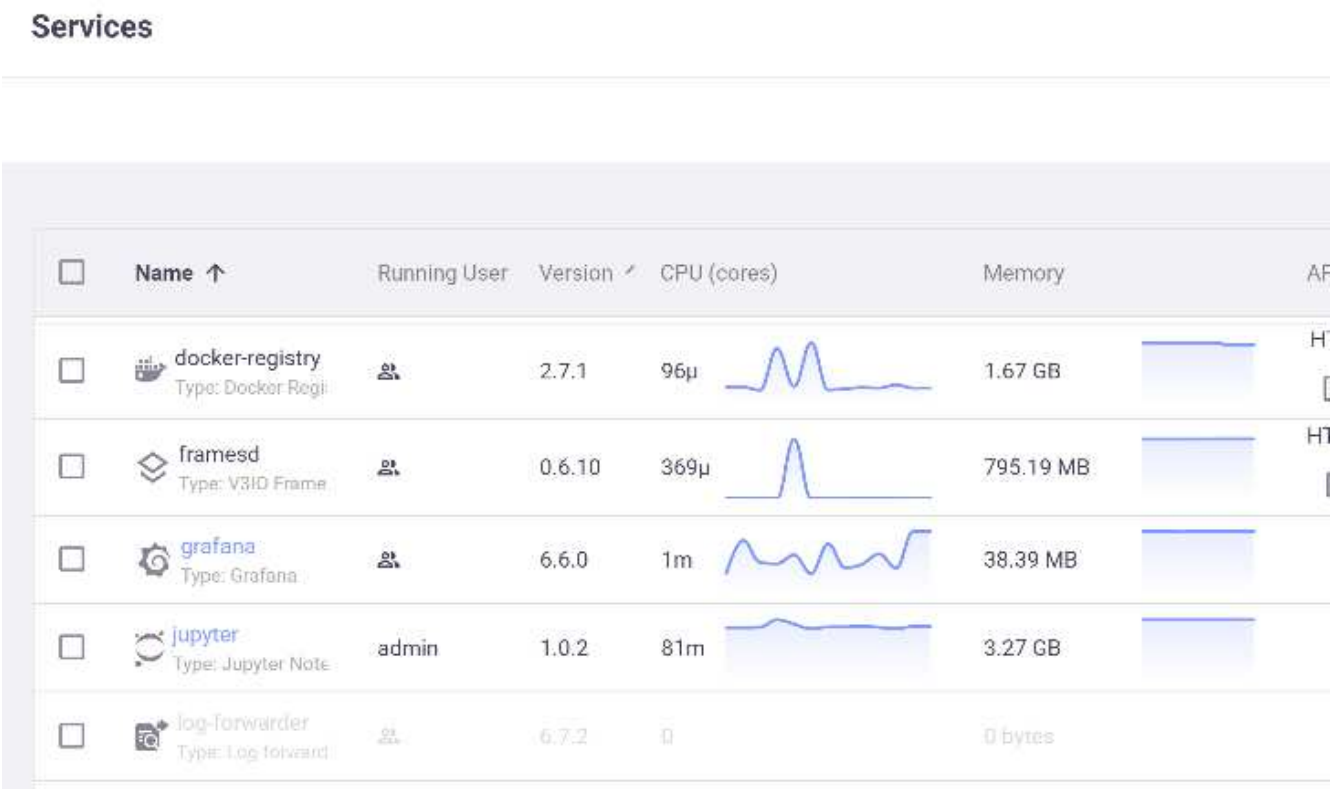
"Als Nächstes: Deploy Grafana Dashboard"

# Implementierung Von Grafana Dashboard

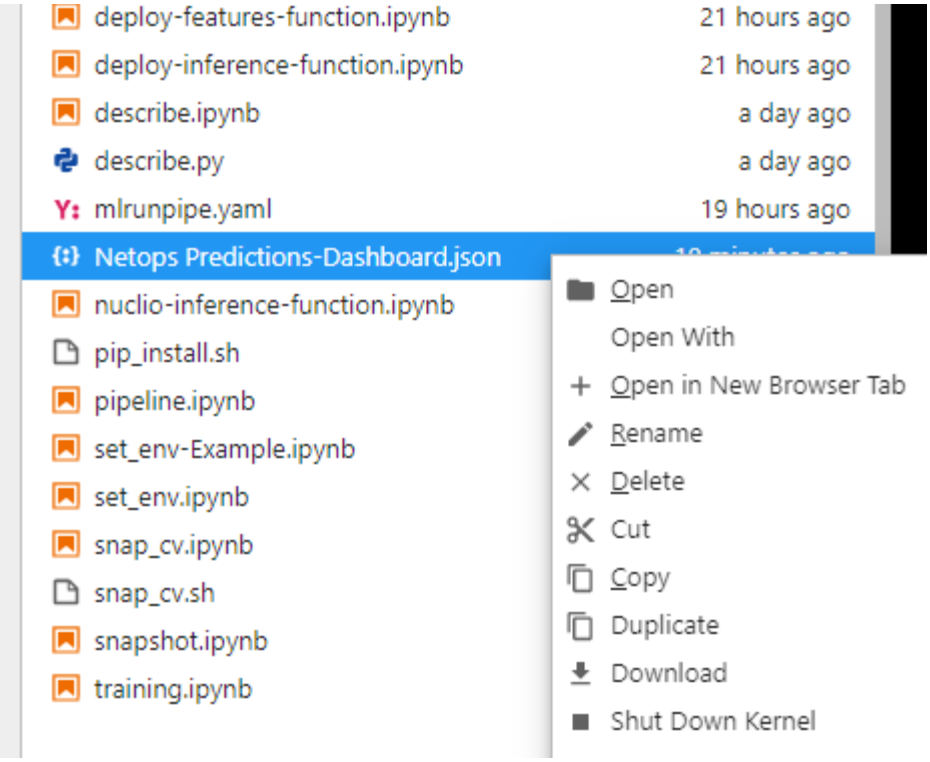
Wenn alles bereitgestellt ist, kommen wir Rückschlüsse auf neue Daten. Die Modelle prognostizieren den Ausfall der Netzwerkgeräte. Die Ergebnisse der Vorhersage werden in einer Iguazio-Timeseries-Tabelle gespeichert. Sie können die Ergebnisse mit Grafana in der in die Sicherheits- und Datenzugriffsrichtlinie von Iguazio integrierten Plattform visualisieren.

Sie können das Dashboard bereitstellen, indem Sie die bereitgestellte JSON-Datei in die Grafana-Schnittstellen im Cluster importieren.

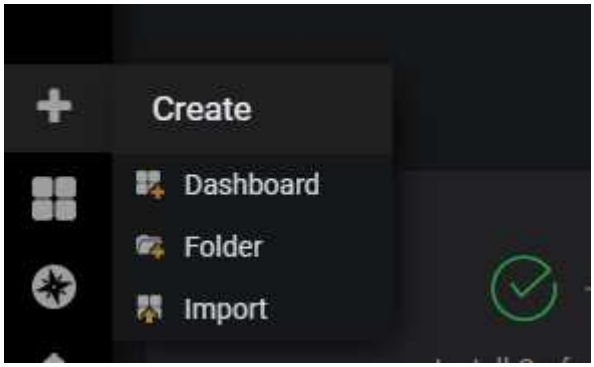
1. Um zu überprüfen, ob der Grafana-Service ausgeführt wird, suchen Sie unter Services.



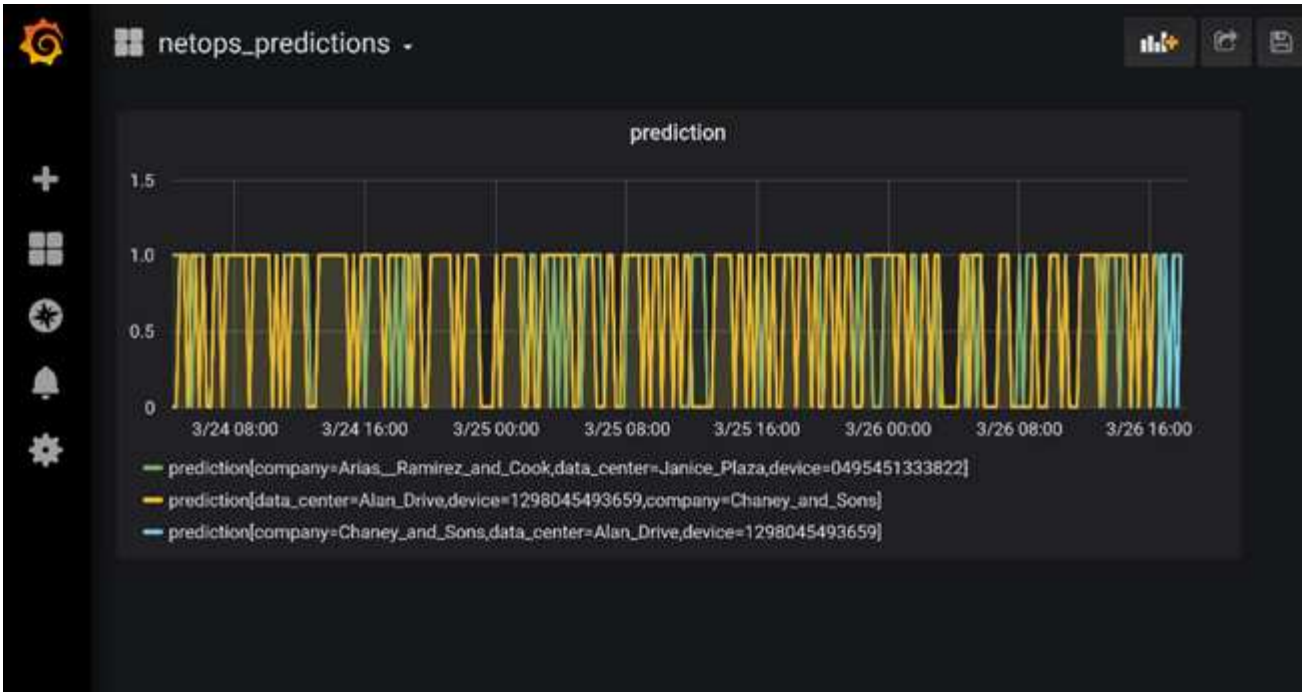

2. Wenn diese nicht vorhanden ist, stellen Sie eine Instanz im Abschnitt „Services“ bereit:
   a. Klicken Sie Auf Neuer Dienst.
   b. Wählen Sie Grafana aus der Liste aus.
   c. Akzeptieren Sie die Standardeinstellungen.
   d. Klicken Sie Auf Weiter.
   e. Geben Sie Ihre Benutzer-ID ein.
   f. Klicken Sie Auf Dienst Speichern.
   g. Klicken Sie oben auf Änderungen übernehmen.
3. Laden Sie die Datei herunter, um das Dashboard bereitzustellen `NetopsPredictions-Dashboard.json` Über die Jupyter-Schnittstelle.




4. Öffnen Sie Grafana im Abschnitt Services, und importieren Sie das Dashboard.




5. Klicken Sie Auf Hochladen `*.json` Datei und wählen Sie die Datei aus, die Sie zuvor heruntergeladen haben (`NetopsPredictions-Dashboard.json`). Das Dashboard wird nach Abschluss des Uploads angezeigt.




## Funktion „Bereinigen“ Bereitstellen

Wenn Sie eine Menge Daten generieren, ist es wichtig, die Dinge sauber und organisiert zu halten. Stellen Sie dazu die Bereinigungsfunktion mit dem bereit `cleanup.ipynb` Notebook.

## Vorteile

NetApp und Iguazio beschleunigen und vereinfachen die Implementierung von KI- und ML-Applikationen. Dazu bauen sie in wichtigen Frameworks wie Kubeflow, Apache Spark und TensorFlow sowie Orchestrierungs-Tools wie Docker und Kubernetes auf. Durch die Vereinheitlichung der End-to-End-Datenpipeline reduzieren NetApp und Iguazio die Latenz und Komplexität moderner Computing-Workloads, wodurch die Lücke zwischen Entwicklung und Betrieb geschlossen wird. Data Scientists können Abfragen zu großen Datensätzen durchführen und Daten und algorithmische Modelle während der Trainingsphase sicher mit autorisierten Benutzern teilen. Sobald die Container-Modelle für die Produktion bereit sind, können Sie sie ganz einfach von Entwicklungsumgebungen in betriebliche Umgebungen verschieben.

"Weiter: Fazit"

**Copyright-Informationen**

Copyright © 2023 NetApp. Alle Rechte vorbehalten. Gedruckt in den USA. Dieses urheberrechtlich geschützte Dokument darf ohne die vorherige schriftliche Genehmigung des Urheberrechtsinhabers in keiner Form und durch keine Mittel – weder grafische noch elektronische oder mechanische, einschließlich Fotokopieren, Aufnehmen oder Speichern in einem elektronischen Abrufsystem – auch nicht in Teilen, vervielfältigt werden.

Software, die von urheberrechtlich geschütztem NetApp Material abgeleitet wird, unterliegt der folgenden Lizenz und dem folgenden Haftungsausschluss:

DIE VORLIEGENDE SOFTWARE WIRD IN DER VORLIEGENDEN FORM VON NETAPP ZUR VERFÜGUNG GESTELLT, D. H. OHNE JEGLICHE EXPLIZITE ODER IMPLIZITE GEWÄHRLEISTUNG, EINSCHLIESSLICH, JEDOCH NICHT BESCHRÄNKT AUF DIE STILLSCHWEIGENDE GEWÄHRLEISTUNG DER MARKTGÄNGIGKEIT UND EIGNUNG FÜR EINEN BESTIMMTEN ZWECK, DIE HIERMIT AUSGESCHLOSSEN WERDEN. NETAPP ÜBERNIMMT KEINERLEI HAFTUNG FÜR DIREKTE, INDIREKTE, ZUFÄLLIGE, BESONDERE, BEISPIELHAFTE SCHÄDEN ODER FOLGESCHÄDEN (EINSCHLIESSLICH, JEDOCH NICHT BESCHRÄNKT AUF DIE BESCHAFFUNG VON ERSATZWAREN ODER -DIENSTLEISTUNGEN, NUTZUNGS-, DATEN- ODER GEWINNVERLUSTE ODER UNTERBRECHUNG DES GESCHÄFTSBETRIEBS), UNABHÄNGIG DAVON, WIE SIE VERURSACHT WURDEN UND AUF WELCHER HAFTUNGSTHEORIE SIE BERUHEN, OB AUS VERTRAGLICH FESTGELEGTER HAFTUNG, VERSCHULDENSUNABHÄNGIGER HAFTUNG ODER DELIKTSHAFTUNG (EINSCHLIESSLICH FAHRLÄSSIGKEIT ODER AUF ANDEREM WEGE), DIE IN IRGENDEINER WEISE AUS DER NUTZUNG DIESER SOFTWARE RESULTIEREN, SELBST WENN AUF DIE MÖGLICHKEIT DERARTIGER SCHÄDEN HINGEWIESEN WURDE.

NetApp behält sich das Recht vor, die hierin beschriebenen Produkte jederzeit und ohne Vorankündigung zu ändern. NetApp übernimmt keine Verantwortung oder Haftung, die sich aus der Verwendung der hier beschriebenen Produkte ergibt, es sei denn, NetApp hat dem ausdrücklich in schriftlicher Form zugestimmt. Die Verwendung oder der Erwerb dieses Produkts stellt keine Lizenzierung im Rahmen eines Patentrechts, Markenrechts oder eines anderen Rechts an geistigem Eigentum von NetApp dar.

Das in diesem Dokument beschriebene Produkt kann durch ein oder mehrere US-amerikanische Patente, ausländische Patente oder anhängige Patentanmeldungen geschützt sein.

ERLÄUTERUNG ZU „RESTRICTED RIGHTS“: Nutzung, Vervielfältigung oder Offenlegung durch die US-Regierung unterliegt den Einschränkungen gemäß Unterabschnitt (b)(3) der Klausel „Rights in Technical Data – Noncommercial Items“ in DFARS 252.227-7013 (Februar 2014) und FAR 52.227-19 (Dezember 2007).

Die hierin enthaltenen Daten beziehen sich auf ein kommerzielles Produkt und/oder einen kommerziellen Service (wie in FAR 2.101 definiert) und sind Eigentum von NetApp, Inc. Alle technischen Daten und die Computersoftware von NetApp, die unter diesem Vertrag bereitgestellt werden, sind gewerblicher Natur und wurden ausschließlich unter Verwendung privater Mittel entwickelt. Die US-Regierung besitzt eine nicht ausschließliche, nicht übertragbare, nicht unterlizenzierbare, weltweite, limitierte unwiderrufliche Lizenz zur Nutzung der Daten nur in Verbindung mit und zur Unterstützung des Vertrags der US-Regierung, unter dem die Daten bereitgestellt wurden. Sofern in den vorliegenden Bedingungen nicht anders angegeben, dürfen die Daten ohne vorherige schriftliche Genehmigung von NetApp, Inc. nicht verwendet, offengelegt, vervielfältigt, geändert, aufgeführt oder angezeigt werden. Die Lizenzrechte der US-Regierung für das US-Verteidigungsministerium sind auf die in DFARS-Klausel 252.227-7015(b) (Februar 2014) genannten Rechte beschränkt.

**Markeninformationen**

NETAPP, das NETAPP Logo und die unter http://www.netapp.com/TM aufgeführten Marken sind Marken von NetApp, Inc. Andere Firmen und Produktnamen können Marken der jeweiligen Eigentümer sein.